

OAuth 2.0 Authentication Framework with Context-based Risk Assessment for AI Chatbot-based RPA

Soonhong Kwon
Dept. of Computer and Information
Security & Convergence Engineering
for Intelligent Drone
Sejong University
Seoul, Republic of Korea
soonhong@pel.sejong.ac.kr

Wooyoung Son
Protocol Engineering Lab.
Sejong University
Seoul, Republic of Korea
wooyoung@pel.sejong.ac.kr

Jong-Hyoun Lee
Dept. of Computer and Information
Security & Convergence Engineering
for Intelligent Drone
Sejong University
Seoul, Republic of Korea
jonghyouk@sejong.ac.kr

Abstract

In response to the growing demand for service automation and 24/7 availability, many organizations are adopting Robotic Process Automation (RPA) technology. However, RPA alone does not ensure complete service automation. To address this, there has been a recent trend to combine AI chatbot technology with RPA to achieve fully automated services that meet the requirements for continuous availability. Despite these advancements, attacks targeting AI chatbots are on the rise, underscoring the need for robust authentication and access control measures. Unfortunately, research and development in this area remain insufficient. To address this gap, this paper proposes an OAuth 2.0 integrated authentication framework, utilizing context-based risk assessment for AI chatbot-based RPA systems. The framework leverages decentralized identity technology to enable identity verification with minimal information and assesses the user's risk level based on context, granting appropriate access rights through policy comparison. Notably, our context-based risk assessment approach has been shown to correctly determine user access rights with an accuracy of approximately 94.4%, ensuring a secure and reliable environment for users.

CCS Concepts

• Security and privacy → Access control.

Keywords

Context based Authentication, DID, OAuth 2.0, AI Chatbot based RPA

ACM Reference Format:

Soonhong Kwon, Wooyoung Son, and Jong-Hyoun Lee. 2024. OAuth 2.0 Authentication Framework with Context-based Risk Assessment for AI Chatbot-based RPA. In *2024 International Conference on Intelligent Computing and its Emerging Application (ICEA 2024)*, November 28–29, 2024, Tokyo, Japan. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3732437.3732762>



This work is licensed under a Creative Commons Attribution International 4.0 License.

ICEA 2024, Tokyo, Japan

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1166-4/24/11

<https://doi.org/10.1145/3732437.3732762>

1 Introduction

As the demand for 24/7 service—meaning 24 hours a day, 7 days a week—has emerged, service automation has become a critical requirement. Among the technologies gaining attention to meet these demands is Robotic Process Automation (RPA). RPA involves using virtual software robots to handle structured, repetitive tasks, and it has recently been adopted not only by businesses but also by public institutions to improve operational efficiency. However, relying solely on RPA is not enough to achieve full-service automation. Consequently, there has been a growing trend to combine AI Chatbot technology with RPA to create fully automated services while also meeting the requirements for 24/7 availability [1].

However, with the recent surge in attacks targeting AI Chatbots, proper authentication and access control have become necessary. Unfortunately, current systems fail to adequately address these concerns [3][4].

In response, this paper proposes an OAuth 2.0 integrated authentication framework using context-based risk assessment to create a secure working environment for users leveraging AI Chatbot-based RPA systems. The proposed framework utilizes Decentralized Identifier (DID) to enable identity verification with minimal information, while assessing user risk based on context to ensure access control tailored to specific tasks. By comparing the user's context to predefined policies, the framework grants appropriate access rights.

The structure of this paper is as follows: In Section 2, we describe OAuth 2.0, which forms the foundation of the proposed framework. Section 3 presents the architecture, message flow, and key algorithms of the framework. Section 4 analyzes the accuracy of the results, focusing on how access rights are granted after evaluating the risk and calculating the score prior to issuing an access token. Finally, Section 5 concludes the paper.

2 OAuth 2.0

OAuth 2.0 has recently become one of the most prominent delegation-based authentication technologies. Delegation-based authentication allows a user to delegate their authentication authority to a third party. Typically, OAuth 2.0 involves four key entities: the user (Resource Owner), the client, the Authorization Server, and the Resource Server. The user is the actual owner of the resource, while the client is an entity that accesses the Authorization and Resource Servers on behalf of the Resource Owner. Figure 1 illustrates the main process of delegation-based authentication using OAuth 2.0 [2].

The process shown in Figure 1 illustrates the basic flow of the authorization code grant type, which is one of the core methods in OAuth 2.0. First, the user sends a request to the client to access a resource. At this point, the user specifies the response type as code in the request. The client then requests an authorization code from the Authorization Server. The parameters included in this request are the client ID, `redirect_url`, and `response_type=code`. Upon receiving the request from the client, the Authorization Server presents a login popup to the user, who completes the login process. If the login is successful, the Authorization Server sends the authorization code back to the client.

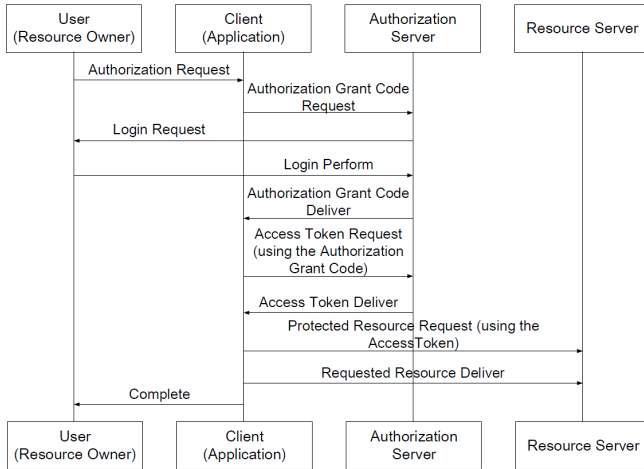


Figure 1: OAuth 2.0 Operation Process

After receiving the authorization code from the Authorization Server, the client must obtain an access token to access the Resource Server. The client requests the issuance of the access token based on the authorization code, and the main parameters in this request include the client ID, client secret, `redirect_url`, and `response_type=code`. The Authorization Server verifies the authorization code provided in the request and, upon successful verification, issues the requested access token to the client.

Once the client receives the access token from the Authorization Server, it makes a request to the Resource Server for the protected resource, using the access token along with the user's original request. The Resource Server verifies the access token and, upon validation, provides the requested resource to the client. Finally, the client delivers the requested resource to the user.

This process allows the user to access the service and utilize resources without directly inputting their credentials. As a result, this technology enables proper access control and reduces the risk of exposing personal or sensitive information during the authentication process.

3 Proposed Framework

Examining the security threats reported in AI Chatbots and RPA systems to date, we find that AI Chatbots have faced issues such as data breaches, data theft, data leaks, jailbreak attacks, and vulnerabilities related to personal information protection. These arise

when sensitive data stored within AI Chatbots becomes accessible, leading to potential leaks of sensitive information or violations of personal privacy.

Additionally, RPA systems are facing challenges related to access control, including unauthorized access and the manipulation or misuse of critical data by internal employees.

This indicates that RPA systems based on AI Chatbots are vulnerable to all of these security threats. To address such issues, Privileged Access Management (PAM) solutions have been adopted to manage and control bots. However, this approach implies that sensitive information still resides within the system and fails to consider factors such as user type and situational context, thereby limiting the effectiveness of timely access management.

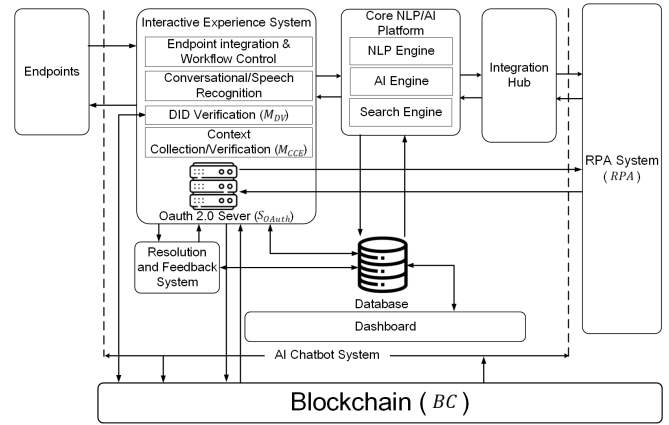


Figure 2: Proposed Framework

In this section, we propose an OAuth 2.0 integrated authentication framework that leverages a context-based risk assessment method to enable timely access control tailored to user types and their surrounding environments. The framework also incorporates DID-based identity authentication technology to perform identity verification using only a minimal amount of personal information. Figure 2 illustrates the OAuth 2.0 integrated authentication framework utilizing context-based risk assessment in AI Chatbot-based RPA systems.

The proposed framework is divided into three main stages: user authentication, risk assessment, and access token issuance and task execution. The primary objectives of each stage are as follows.

In the first stage of user authentication, we aimed to structure it so that authentication is performed based on minimal user data (e.g., personal or sensitive information). Traditional authentication methods, such as ID/password-based or certificate-based authentication, may imply that such information could be stored in the AI chatbot's database. This implies that if the AI chatbot system is compromised by an attacker, personal or sensitive information could be leaked or misused. Therefore, the primary goal of the user authentication stage is to propose a method that enables authentication based on minimal data. To achieve this, the proposed framework aims to leverage DID technology, specifically utilizing the DID:web method to implement user authentication. In the case of DID:web, one key feature is that identity verification can be achieved by storing the DID Document on the web, rather than on a blockchain. The user

provides the AI chatbot with a DID based on the DID:web method, and the chatbot then identifies the DID Document uploaded on the web. By using the public key from the document, it verifies the user's identity, allowing authentication to be carried out with minimal user information.

Table 1: Notation

Notation	Description
Cli	Client
M_{DV}	DID Verification Module
$SOAuth$	OAuth 2.0 Server
M_{CCE}	Context Collection/Evaluation Module
RPA	Robotic Process Automation
BC	Blockchain
C_{Work}	Task to be Performed by the Client
DID_C	Client DID
Doc_C	Client DID Document
M_C	Client Message
M_{Signed}	Message Signed with Client's Private Key
K_C^{Pri}	Client Private Key
K_C^{Pub}	Client Public Key
$AuthorizationCode$	Authorization Code
RA	Risk Assessment Score
RA_{Result}	Risk Assessment Result
C_1	Location Context
C_2	Connection Type Context
C_3	Time Zone Context
C_4	User Activity Sensitivity Context
CI_1	Location Context Weight Value
CI_2	Connection Type Context Weight Value
CI_3	Time Zone Context Weight Value
CI_4	User Activity Sensitivity Context Weight Value
CI_i	Context Weight Value
$W_{i,j}$	Weight Value
$Policy$	Policy
$AccessToken$	Access Token
ACM	Access Control Matrix
$Accuracy$	Accuracy of Appropriate Access Permission Granted
$Result$	Task Execution Result
$Sign(x,y,z)$	Signing y with x to Generate z
$Lookup(x,y)$	Querying y Based on x
$Extract(x,y)$	Extracting y from x
$Verify(x,y)$	Verifying y with x
$Collect(x)$	Collecting x
$Calculate(x,y)$	Calculating y Based on x
$Check(x,y)$	Inspecting y Based on x
$Issue(x)$	Issuing x
$Perform(x,y)$	Performing y Based on x

Secondly, in the risk assessment stage, context information is collected when the user makes a request, and the risk is evaluated to ensure that appropriate permissions can be delegated to the AI chatbot. Attackers could impersonate the user or steal personal information to gain access to the AI chatbot. This implies that attackers may perform sensitive operations within the chatbot or misuse the user's personal and sensitive information to carry out abnormal actions. More specifically, even if the user has not actually performed any abnormal actions, they could still be held accountable for them. Therefore, the proposed framework collects context information from the moment the user makes a request and evaluates the risk based on that context. The evaluated risk is then used by the OAuth server's internal policy to issue access tokens.

Thirdly, in the stage of access token issuance and task execution, the goal is to differentiate policies based on the risk level, ensuring that the appropriate access token is issued and used according to the assigned permissions. The risk assessed in the previous stage is

used by the OAuth 2.0 server's policy to issue access tokens with permissions that are appropriate to the situation. As a result, even if an attacker tries to request data using the user's personal or sensitive information, the risk assessment process helps reduce the likelihood of the attacker's intended outcome being realized.

Algorithm 1 DID-based Client Authentication

```

1: begin
2: input  $C_{Work}$ ,  $DID_C$ ,  $K_C^{Pri}$ ,  $AI\_Chatbot\_System$ 
3:  $M_{Signed} \leftarrow 0$ 
4:  $cli\_work \leftarrow C_{Work}$ 
5:  $did\_request \leftarrow M_{DV}.request(DID_C)$ 
6:  $M_C \leftarrow$  "Client's message content"
7:  $M_{Signed} \leftarrow sign(M_C, K_C^{Pri})$ 
8:  $M_{DV}.receive(M_{Signed}, DID_C)$ 
9:  $Doc_C \leftarrow request\_did\_document\_via\_web(DID_C)$ 
10:  $K_C^{Pub} \leftarrow Doc_C.public\_key$ 
11: if  $verify(M_{Signed}, K_C^{Pub})$  then
12:   print "Identity verification successful!"
13:    $K \leftarrow$  "Valid"
14: else
15:   print "Identity verification failed!"
16:    $K \leftarrow$  "Invalid"
17: end if
18: output  $K$ 
19: end
    
```

The detailed operational process is explained in Figure 3, based on Table 1, as follows.

- (1) Cli requests C_{Work} from the AI Chatbot system.
- (2) The AI Chatbot system identifies C_{Work} received from Cli through the interactive experience system and core NLP/AI platform, and requests DID_C from Cli via M_{DV} .
- (3) Cli generates M_{Signed} by signing M_C using K_C^{Pri} . This data is used to verify DID_C .
- (4) Cli sends M_{Signed} , generated in step (3), along with DID_C to M_{DV} .
- (5) M_{DV} searches internally for Doc_C based on DID_C received in step (4).
- (6) If M_{DV} cannot identify Doc_C internally in step (5), it requests Doc_C from BC based on DID_C .
- (7) BC identifies Doc_C associated with DID_C and sends it to M_{DV} .
- (8) M_{DV} extracts K_C^{Pub} from Doc_C received from BC .
- (9) M_{DV} verifies M_{Signed} using K_C^{Pub} extracted in step (8). If the verification is successful, DID_C is identified as valid, completing the identity verification process.
- (10) Afterward, M_{DV} requests the $AuthorizationCode$ from $SOAuth$.
- (11) $SOAuth$ generates a login popup and sends it to Cli , requesting a login.
- (12) Cli reviews the login popup received in step (10) and proceeds with the login process.
- (13) After confirming that Cli has successfully completed the login process, $SOAuth$ sends $AuthorizationCode$ to Cli .
- (14) $SOAuth$ then requests the RA_{Result} from M_{CCE} .

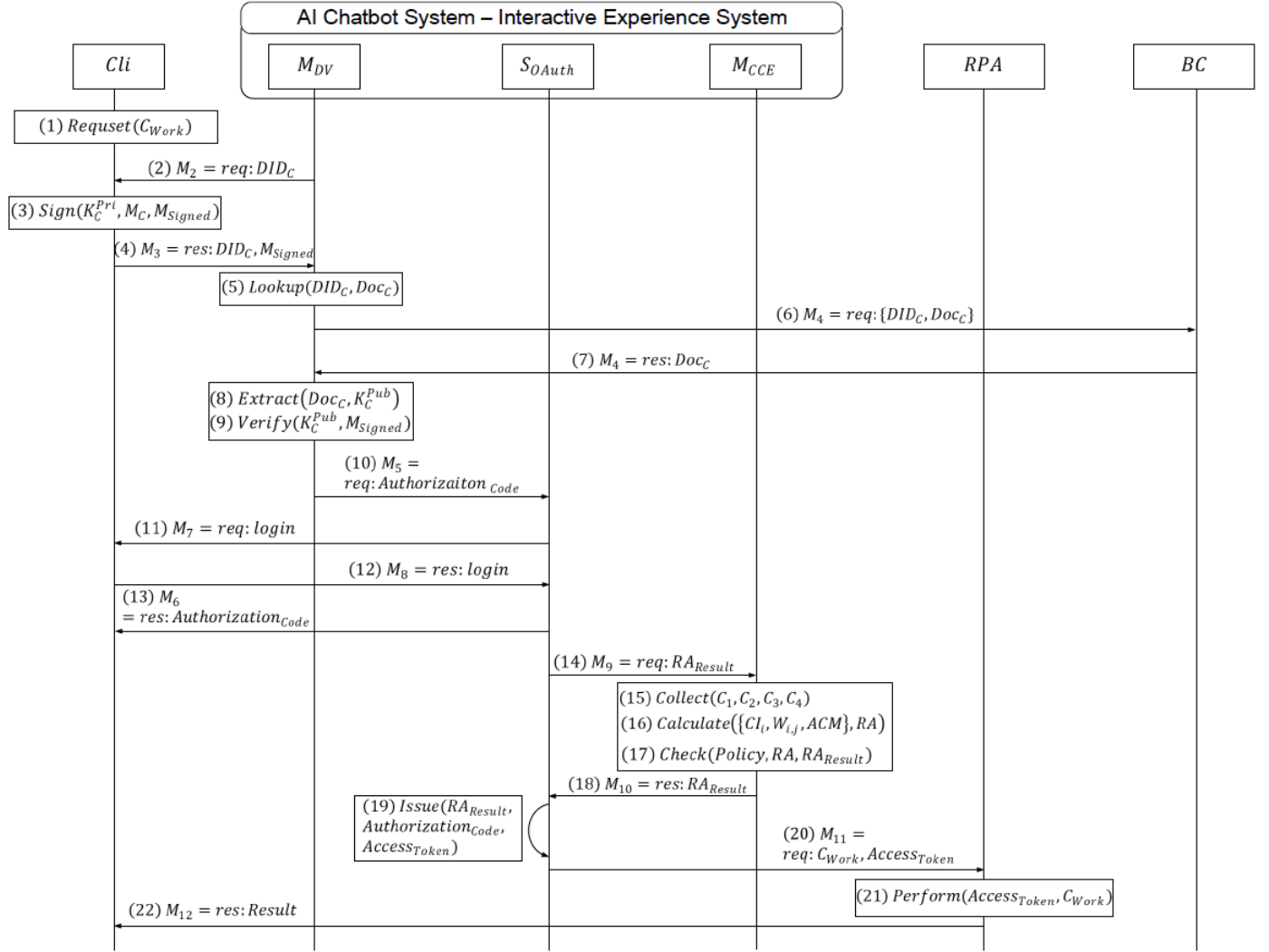


Figure 3: Detailed Operation Process of the Proposed Framework

- (15) M_{CCE} collects information on C_1, C_2, C_3 , and C_4 .
- (16) M_{CCE} calculates RA by performing operations on CI_i and $W_{i,j}$.
- (17) M_{CCE} checks RA calculated in step (16) against $Policy$ to determine the access control level and generates RA_{Result} .
- (18) M_{CCE} sends RA_{Result} generated in step (17) to $SOAuth$.
- (19) $SOAuth$ issues $AccessToken$ based on RA_{Result} received in step (18) by using $AuthorizationCode$.
- (20) $SOAuth$ sends $AccessToken$ and C_{Work} to RPA in step (19).
- (21) RPA performs C_{Work} based on $AccessToken$.
- (22) After completing C_{Work} , RPA delivers $Result$ to Cli .

In Figure 3, steps (1) to (13) correspond to the user authentication stage, with steps (1) to (9) representing the identity verification phase based on DID_C . In this framework, the DID:web method is used for implementation, and the key algorithm can be found in Algorithm 1.

As explained in Algorithm 1, the process begins with Cli requesting C_{Work} . Following this, the AI Chatbot System requests DID_C for Cli 's identity verification, and Cli sends both DID_C and M_{Signed} , which is signed with K_C^{Pri} , to MDV .

Afterward, MDV checks Doc_C , which is uploaded on the web based on DID_C , and extracts K_C^{Pub} to verify M_{Signed} . If the verification is successful, Cli 's identity authentication is completed. This process is illustrated in Figure 4.

In Figure 3, steps (14) to (18) correspond to the risk assessment stage. This process begins when $SOAuth$ requests the RA_{Result} from M_{CCE} . When Cli requests C_{Work} , M_{CCE} collects context information (e.g., C_1, C_2, C_3, C_4) and stores it in JSON file format.

Afterward, based on the contents of the JSON file, RA is calculated using $CI_i, W_{i,j}$, and ACM according to Equation (1). M_{CCE} derives the final RA_{Result} based on the Policy outlined in Equation (2).

```

AI Chatbot server is running on http://localhost:3000
Received DID: did:web:soonding.github.io:client_did
Received message: Client's message content
Received signature: 0xc3f3624949962c626af09d0eabe966043f33b9e0206286f53b8c359a9ed44be4165c8e
cb0a5f78c1438da9a7661348579d83536920205de26a70064302f9c921b
Resolved DID Document: {
  "@context": "https://www.w3.org/ns/did/v1",
  "id": "did:web:soonding.github.io:client_did",
  "verificationMethod": [
    {
      "id": "did:web:soonding.github.io:client_did#owner",
      "type": "JsonWebKey2020",
      "controller": "did:web:soonding.github.io:client_did",
      "publicKeyJwk": {
        "kty": "EC",
        "crv": "secp256k1",
        "x": "uXyY0mD0+T66h1XRID8FT1UK/GL9BebFA3XheVRVjgI=",
        "y": "J2dRKGjdzRytIdY397QmQwUtduaUvgIcsRKXq4bA="
      }
    }
  ],
  "authentication": [
    {
      "id": "did:web:soonding.github.io:client_did#owner"
    }
  ],
  "assertionMethod": [
    {
      "id": "did:web:soonding.github.io:client_did#owner"
    }
  ]
}
Message hash (server): 0f12c782d2250fd3ffdf8eala03e5a784e72ef34079a9faa80ec39423eb65c2d
Recovered publicKey: 0xb97c98d260f4f93ebale25d1203f054f550afc62fd05e6c50375e17954558e022767
6b457823773458b62758dfded3990426c14b5db9a52f80872c44ac6aeb0
Expected publicKeyX: b97c98d260f4f93ebale25d1203f054f550afc62fd05e6c50375e17954558e02
Expected publicKeyY: 27676b457823773458b62758dfded3990426c14b5db9a52f80872c44ac6aeb0
Identity Verification Completed!!
    
```

Figure 4: Identity Verification using DID

```

2024-09-06 01:36:18,904 - INFO - Press CTRL+C to quit
2024-09-06 01:36:51,511 - INFO - Token issuance requested
2024-09-06 01:36:51,511 - INFO - Context: location=C1_1, connection_type=C2_1,
time=C3_1, activity=C4_1
2024-09-06 01:36:51,511 - INFO - Risk score: 4
2024-09-06 01:36:51,511 - WARNING - Limited access
2024-09-06 01:36:51,511 - INFO - 127.0.0.1 - - [06/Sep/2024 01:36:51] "POST /oau
th/token HTTP/1.1" 200 -
    
```

Figure 5: Context-based Risk Assessment for Access Control

M_{CCE} then sends the RA_{Result} to $SOAuth$ to ensure that the appropriate permissions are granted for generating $AccessToken$, thereby laying the foundation for issuing $AccessToken$ with the proper permissions. Figure 5 shows the screen where RA_{Result} is generated through the risk assessment process.

$$RA = \sum ACM \times \sum_{i=1}^4 \sum_{j=1}^4 CI_i \times W_{i,j} \quad (1)$$

$$RA_{Result} = f(RA, \theta) = \begin{cases} \text{Full Access} & \text{if } RA < \theta \\ \text{Limited Access} & \text{if } RA = \theta \\ \text{Access Denied} & \text{if } RA > \theta \end{cases} \quad (2)$$

```

2024-09-05 02:31:10 INFO root - Rasa server is up and running.
Bot loaded. Type a message and press enter (use '/stop' to exit):
Your input -> /request_rpa_job

Requesting RPA job execution with OAuth 2.0 token...
🔔 RPA Job Request Received 🔔
🔔 **Limited Access Token**: limited_access_token
🔔 **Reason for Limited Access**:
Location is marked as high-risk.
Connection type is less secure.
Request was made outside of regular hours.
Suspicious activity detected.
🔔 **Token Value**: od5txFTgIvEAK9ihMu2QFjDZtH1CIL
RPA Response: Limited RPA task executed.
    
```

Figure 6: Access Token Issuance and Client Request Execution

Steps (19) to (22) in Figure 3 correspond to the stage of issuing $AccessToken$ and performing C_{Work} . $SOAuth$ uses RA_{Result} received

from M_{CCE} and $AuthorizationCode$ to issue an $AccessToken$ that corresponds to RA_{Result} . Then, it sends $AccessToken$ and C_{Work} to RPA , which performs C_{Work} in compliance with $AccessToken$, and afterward, delivers $Result$ to CLI . Figure 6 shows the result screen where this series of processes is carried out in the AI Chatbot system.

The proposed context-based risk assessment OAuth 2.0 integrated authentication framework leverages DID technology to perform identity verification with minimal information, mitigating the risk of personal data leakage even if the AI chatbot system is compromised by an attacker. Additionally, by combining OAuth 2.0 with context-based risk assessment, the framework ensures that specific task requests are carried out based on proper authorization, reducing the likelihood of abnormal actions by attackers.

4 Analysis

In this section, we aim to demonstrate the accuracy of the results where permissions are granted based on a risk assessment before issuing $AccessToken$ in response to CLI 's request through the proposed authentication framework. The framework continuously collects context information during the authentication process and performs a risk assessment based on that information before issuing $AccessToken$. The evaluation criteria used for the risk assessment are outlined in Table 2.

Table 2: Risk Assessment Criteria

Context	Weight Value	Detailed Context	Weight Value
Location Information	CI_1 (0.60)	Internal	$W_{1,1}$ (0.69)
		External	$W_{1,2}$ (0.75)
Connection Type	CI_2 (0.52)	Internal	$W_{2,1}$ (0.53)
		Network	
		Remote Access	$W_{2,2}$ (0.57)
Time Zone	CI_3 (0.43)	Working Hours	$W_{3,1}$ (0.41)
		Outside	
User Activity Sensitivity	CI_4 (0.33)	Working Hours	$W_{3,2}$ (0.44)
		General	
		Task	$W_{4,1}$ (0.30)
		Sensitive Task	$W_{4,2}$ (0.32)

In the case of risk assessment, in addition to the collected context information, the score for the user's access rights to the requested task is determined based on Table 4. RA is calculated using Equation (1), based on Table 2 and Table 3.

The weights used in Table 2 and Table 3 for calculating RA are values set to reflect the importance of each context, while also determining the range of the calculated score. This allows for assessing the user's access rights to the requested task.

Based on the collected context information, the maximum and minimum values that can be derived from Table 2 are 1.0412 and 0.9649, respectively. The values of $\sum ACM$ according to access rights

Table 3: ACM Evaluation Criteria

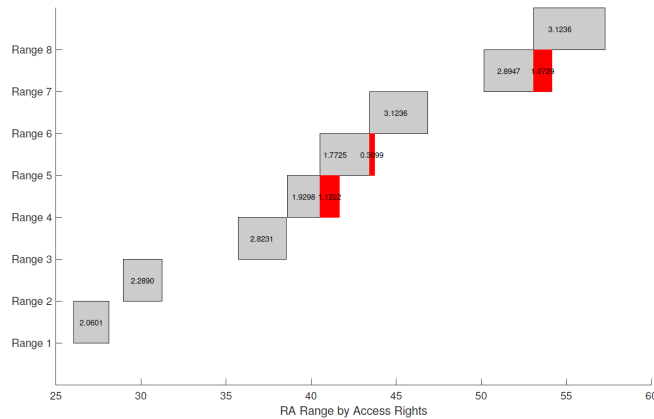
ACM	O	X
read	2.0	5.0
write	10.0	20.0
execute	15.0	30.0

and RA values for each level of access are shown in Table 4, while RA range for each access level is illustrated in Figure 7.

Table 4: $\sum ACM$ Values and RA Scores based on Access Permissions

read	write	execute	$\sum ACM$	RA
O	O	O	27.0	26.0523 - 28.1124
X	O	O	30.0	28.947 - 31.236
O	X	O	37.0	35.7013 - 38.5244
X	X	O	40.0	38.596 - 41.6488
O	O	X	42.0	40.5258 - 43.7304
X	O	X	45.0	43.4205 - 46.854
O	X	X	52.0	50.1748 - 54.1424
X	X	X	55.0	53.0695 - 57.266

As seen in Figure 7, Range 4 and Range 5, as well as Range 5 and Range 6, and Range 7 and Range 8, overlap. For the other Ranges, there is no overlap with different Ranges. Therefore, for RA values within these ranges, the user's access rights are determined based on RA , and the corresponding access token is provided accordingly.

**Figure 7: RA Range for Each Access Permission**

Due to the overlapping sections between two consecutive ranges, the probability of accurately determining the user's access rights for RA within the overlapping portion is 50%. As shown in Figure 7, the total length of the range that can be derived from RA is 22.5214. Of this, the length of the overlapping sections between

consecutive ranges is 2.505, while the non-overlapping portion is 20.0164. Accordingly, the accuracy of correctly identifying the user's access rights based on RA is represented by Equation (3).

$$\text{Accuracy} = \frac{2.505}{22.5214} \times 0.5 + \frac{20.0164}{22.5214} \times 1 = 0.9444 \quad (3)$$

This shows that the user's access rights can be correctly determined with approximately 94.44% accuracy based on RA . After verifying whether the user's RA falls within a specific range, permissions can be granted differentially depending on the user's RA within that range. The closer RA is to the lower end of the range, the more permissions the user receives. By identifying the user's access rights in this way and granting access permissions based on the relative position of RA within the assigned range, the access rights for the requested task can be more finely adjusted.

5 Conclusion

This paper proposed the OAuth 2.0 integrated authentication framework using context-based risk assessment to create a secure and trustworthy environment where users can access AI chatbot-based RPA systems with minimal information, while enabling access control based on user context. The framework leverages DID technology to ensure identity verification with minimal data and combines OAuth 2.0 with context-based authentication to assign appropriate access rights according to the user's situation. This approach helps prevent the misuse or abuse of permissions and reduces the likelihood of attackers performing malicious actions. Notably, the context-based risk assessment method within the proposed framework demonstrated an accuracy of approximately 94.44% in determining user access rights, ensuring a reliable and secure environment. However, a comprehensive security analysis of the AI chatbot-based RPA system built on this framework has yet to be conducted. Future work will focus on performing this security analysis to further contribute to the development of a highly reliable and efficient AI chatbot-based work environment.

Acknowledgments

This research was supported by Korea Institute of Science and Technology Information(KISTI).(No. (KISTI)J24JR007241). This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) under the metaverse support program to nurture the best talents (IITP-2023-RS-2023-00254529) grant funded by the Korea government (MSIT).

References

- [1] Terrence Chong et al. 2021. AI-chatbots on the services frontline addressing the challenges and opportunities of agency. *Journal of Retailing and Consumer Services* 63 (2021), 102735. doi:10.1016/j.jretconser.2021.102735
- [2] Internet Engineering Task Force (IETF). 2012. *The OAuth 2.0 Authorization Framework (RFC 6749)*. Technical Report. Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc6749> Accessed: October 1, 2023.
- [3] Glorin Sebastian. 2023. Do ChatGPT and other AI chatbots pose a cybersecurity risk?: An exploratory study. *International Journal of Security and Privacy in Pervasive Computing (IJSPPC)* 15, 1 (2023), 1–11.
- [4] Yuntao Wang et al. 2023. A survey on ChatGPT: AI-generated contents, challenges, and solutions. *IEEE Open Journal of the Computer Society* (2023). doi:10.1109/OJCS.2023.3249293